VIETNAM NATIONAL UNIVERSITY, HANOI

INTERNATIONAL SCHOOL

Course: Modern Data Mining

FINAL REPORT

Topic: EMPLOYEE ATTRITION & PERFORMANCE OF IBM

Lecturer	:	**Pham Danh Tuyen**	
Course ID	:	**INS4017-01**	
Group	:	**03**	
Member	:	**Do Thanh Van**	**21070790**
		Nguyen Bich Diep	**21070891**
		Ha Thi Hong Anh	**21070345**

Hanoi, June 12st 2025

Acknowledgment

We extend our heartfelt thanks to Vietnam National University (VNU) for the invaluable experiences and knowledge gained during the course ' Modern Data Mining'. The university provided a comprehensive learning environment that facilitated a smooth completion of the course, significantly enriching our understanding of business analytics.

Our sincere appreciation goes to the International School of VNU for their guidance, expertise, and unwavering support throughout the program. The quality education provided by the International School was instrumental in developing our skills and enhancing our analytical abilities.

We also express gratitude to our dear friends for their close companionship, collaboration, and shared learning experiences. Their insights and perspectives added a dynamic and enriching dimension to our educational journey, making it both interesting and rewarding.

To our relatives and family members, your unwavering support has been our pillar of strength. Your encouragement and understanding during the challenging times of this course were crucial in our successful completion.

Reflecting on this educational journey, we are deeply grateful for the collective efforts of VNU, the International School, friends, relatives, and family. Your contributions not only shaped our academic growth but also laid a solid foundation for our future endeavors in the field of business analytics.

We are sincerely appreciative of PhD Pham Danh Tuyen's dedication, which played a pivotal role in the successful completion of this project. His support not only deepened my understanding but also contributed significantly to the broader research in this field.

Table of Contents

1. Introduction 6

1.1. About the Project 6

a. Project Problem 6

b. Solution 6

1.2. Project Objectives, Project Scope 6

a. Project Objectives 6

b. Project Scope 7

2. Data Description and Preprocessing 7

2.1. Dataset Information 7

a. Source of dataset 7

b. Dataset description 8

c. Descriptive statistics 9

2.2. Data Cleaning 11

2.3. Exploratory Data Analysis (EDA) 13

2.4. Data preprocessing 16

a. Check for imbalance in dataset 16

b. Split the data 17

c. Handle data imbalance use SMOTE 17

3. Methodology 18

3.1. Workflow 18

3.2. Techniques Applied 18

3.3. Tools and Libraries Used 20

4. Results 21

4.1. Random Forest model 21

4.2. XGBoost model 22

4.3. KNeighbors Classifier model 23

4.4. Stacking model **24**

4.5. Result summary **25**

5. Conclusion **27**

6. References **30**

List of figures

Figure 1: Dataset first look 8
Figure 2: Import Libraries 9
Figure 3: Display the dataset 9
Figure 4: Identify the data types of columns 10
Figure 5: Random Sample of dataset with only Categorical Feature 10
Figure 6: Mapping 11
Figure 7: Random Sample of dataset with only Numerical Feature 11
Figure 8: Missing value 12
Figure 9: Duplicate records 12
Figure 10: The unique value 13
Figure 11: Features scaling 13
Figure 12: Dataset Preview and Dimension Summary 13
Figure 13: Data information 14
Figure 14: Density Distribution of Age and Years at Company by Attrition Status 15
Figure 15: Heatmap 16
Figure 16: Data describe 16
Figure 17: Visualization 'Attrition' 17
Figure 18: Separate dependent and independent variables 17
Figure 19: Training and test data 18
Figure 20: Flow chart 18
Figure 21: Random forest on dataset 20
Figure 22: KNeighborsClassifier on dataset 20
Figure 23: Random Forest 22
Figure 24: XGBoost 23
Figure 25: KNeighbors Classifier 24
Figure 26: Stacking model 25

List of table

Table 1: Attribute description 8
Table 2: Result of random forest model 21
Table 3: Results of XGBoost model 22
Table 4: Results of KNeighbors Classifier model 23
Table 5: Results of Stacking model 24
Table 6: Result summary 25

1. Introduction

1.1. About the Project

a. Project Problem

Maintaining stability in human resources within the labour market is a critically important issue for any company or organization. High employee turnover can result in significant internal disruptions and damage a company's public image and reputation. The challenges of employee attrition and declining performance continue to be pressing concerns for organizations, as these issues directly affect productivity and operational efficiency. Turnover not only leads to increased costs related to recruitment, onboarding, and retraining, but also negatively influences team morale, workplace culture, and the overall working environment. The ultimate goal for any organization is to reduce turnover, foster employee loyalty and engagement, and build a positive work culture that motivates individuals to perform at their best. To do this, companies must understand the underlying causes of employee departures and performance drops.

b. Solution

Our project aims to support companies in addressing these human resource challenges through a data-driven approach. By leveraging advanced data analysis models, we provide in-depth insights into the key drivers of employee turnover and disengagement. These analyses help organizations uncover the root causes of workforce instability, enabling them to design effective strategies for recruitment, training, and retention. The ultimate objective is to help companies optimize their human resource practices by providing clear, actionable insights that lead to a more stable, motivated, and productive workforce. With the right strategies in place, businesses can enhance employee satisfaction, reduce attrition, and strengthen long-term organizational performance.

1.2. Project Objectives, Project Scope

a. Project Objectives

Employee attrition refers to the gradual reduction of a company's workforce due to voluntary or involuntary departures of employees. So that, we need to understanding employee attrition is important for several reasons:

- Workforce Planning: By analyzing attrition patterns, companies can better predict future staffing needs and plan accordingly to maintain the required workforce.

- Cost Reduction: High employee turnover can be costly for organizations, as they have to frequently hire and train new employees. Understanding attrition can help companies implement strategies to retain valuable employees.
- Organizational Performance: Excessive employee attrition can negatively impact an organization's productivity, institutional knowledge, and overall performance. Addressing attrition can help maintain a stable and experienced workforce.
- Employee Engagement: Examining the reasons behind employee attrition can provide insights into factors that affect employee satisfaction, engagement, and overall well-being within the organization.

b. Project Scope

- Analyse employee attrition patterns using key variables such as age, department, education, job role, satisfaction levels, and work-life balance.
- Use provided HR data to identify relationships between employee characteristics (e.g. income, overtime, tenure) and their likelihood of leaving the company.
- Apply statistical and machine learning techniques to discover hidden patterns influencing turnover and performance.
- Generate insights to support HR in improving retention, job satisfaction, and employee engagement.
- Deliver data visualisations and actionable recommendations to guide strategic workforce planning.

2. Data Description and Preprocessing

2.1. Dataset Information

a. Source of dataset

Overview of this dataset, we have compiled quite a few dataset sources on Kaggle and combined them to create complete data to be able to process and create a movie recommendation model.

Not only that, we also collected a data source about movie reviews to see if it is a negative or positive review. So, our project has 2 main data: movie_data and review_data.

Our dataset is taken from kaggle:
https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

Age	Attrition	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfaction	Gender
41	Yes	Sales	1	2	Life Sciences	2	Female
49	No	Research & Development	8	1	Life Sciences	3	Male
37	Yes	Research & Development	2	2	Other	4	Male
33	No	Research & Development	3	4	Life Sciences	4	Female

HourlyRate	JobInvolvement	JobRole	JobSatisfaction	MaritalStatus	MonthlyIncome	OverTime	PercentSalaryHike
94	3	Sales Executive	4	Single	5993	Yes	11
61	2	Research Scientist	2	Married	5130	No	23
92	2	Laboratory Technician	3	Single	2090	Yes	15
56	3	Research Scientist	3	Married	2909	Yes	11

RelationshipSatisfaction	TotalWorkingYear	WorkLifeBalanc	YearsAtCompany	YearsInCurrentRole
1	8	1	6	4
4	10	3	10	7
2	7	3	0	0
3	8	3	8	7

Figure 1: Dataset first look

This is a synthetic dataset generated by the IBM Data Scientists consisting of 21 rows and 1470 attributes, including categorical and numerical variables. Discover the factors that contribute to employee attrition and ask key questions like "show me a breakdown of distance from home by job role and attrition" or "compare average monthly income by education and attrition".

b. Dataset description

IBM has gathered information on employee satisfaction, income, seniority and some demographics. It includes the data of 1470 employees.

Table 1: Attribute description

Name	**Description**
AGE	Numerical Value
ATTRITION	Employee Leaving The Company (0=No, 1=Yes)
DEPARTMENT	(1=Hr, 2=R&D, 3=Sales)
DISTANCE FROM HOME	Numerical Value - The Distance from Work to Home
EDUCATION	Numerical Value
EDUCATION FIELD	(1=Hr, 2=Life Sciences, 3=Marketing, 4=Medical Sciences, 5=Others, 6= Technical)
ENVIRONMENT SATISFACTION	Numerical Value - Satisfaction with the Environment
GENDER	(1=Female, 2=Male)
HOURLY RATE	Numerical Value - Hourly Salary
JOB INVOLVEMENT	Numerical Value - Job Involvement
JOB ROLE	(1=Hc Rep, 2=Hr, 3=Lab Technician, 4=Manager, 5= Managing Director, 6= Research Director, 7= Research Scientist, 8=Sales Executive, 9= Sales Representative)
JOB SATISFACTION	Numerical Value - Satisfaction With The Job
MARITAL STATUS	(1=Divorced, 2=Married, 3=Single)
MONTHLY INCOME	Numerical Value - Monthly Salary

OVERTIME	(1=No, 2=Yes)
PERCENT SALARY HIKE	Numerical Value - Percentage Increase In Salary
RELATIONS SATISFACTION	Numerical Value - Relations Satisfaction
TOTAL WORKING YEARS	Numerical Value - Total Years Worked
WORK LIFE BALANCE	Numerical Value - Time Spent between Work & Outside
YEARS AT COMPANY	Numerical Value - Total Number of Years at the Company
YEARS IN CURRENT ROLE	Numerical Value -Years in Current Role

c. Descriptive statistics

Before getting into sophisticated visualizations, we want to ensure the quality of our data with Python software. This basic step is critical for gaining a more in-depth understanding of how we should efficiently manage our data in following stages of the project.

```
[ ] import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    import seaborn as sns
    import plotly.express as px
    import warnings
    warnings.filterwarnings('ignore')
```

Figure 2: Import Libraries

```
cd "drive/MyDrive/nguyen/"
```

```
/content/drive/MyDrive/nguyen
```

```
data = pd.read_csv('WA_Fn-UseC_-HR-Employee-Attrition.csv')
data.head()
```

	Age	Attrition	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfaction	Gender	HourlyRate	JobInvolvement	...	JobSatisfaction
0	41	Yes	Sales	1	2	Life Sciences	2	Female	94	3	...	4
1	49	No	Research & Development	8	1	Life Sciences	3	Male	61	2	...	2
2	37	Yes	Research & Development	2	2	Other	4	Male	92	2	...	3
3	33	No	Research & Development	3	4	Life Sciences	4	Female	56	3	...	3
4	27	No	Research & Development	2	1	Medical	1	Male	40	3	...	2

5 rows × 21 columns

Figure 3: Display the dataset

- **Dataset Structure**: The dataset has 1470 rows and 21 columns.

- **Missing Data**: There are no null values in the dataframe.

- **Data type**: There are 7 columns with object type data and 14 columns with integer value data.

```
[ ] # Identify the data types of columns
    column_data_types = data.dtypes

    # Count the numerical and categorical columns
    numerical_count = 0
    categorical_count = 0

    for column_name, data_type in column_data_types.items():
        if np.issubdtype(data_type, np.number):
            numerical_count += 1
        else:
            categorical_count += 1

    # Print the counts
    print(f"There are {numerical_count} Numerical Columns in dataset")
    print(f"There are {categorical_count} Categorical Columns in dataset")
```

```
There are 14 Numerical Columns in dataset
There are 7 Categorical Columns in dataset
```

Figure 4: Identify the data types of columns

- **Numerical data:** 14 Columns
- **Categorical data:** 7 Columns

```
#Random Sample of dataset with only categorical Feature
data.select_dtypes(include='O').sample(5)
```

	Attrition	Department	EducationField	Gender	JobRole	MaritalStatus	OverTime
280	No	Research & Development	Medical	Male	Research Director	Married	Yes
218	No	Sales	Medical	Female	Sales Executive	Single	No
206	No	Research & Development	Life Sciences	Male	Research Scientist	Divorced	Yes
191	No	Research & Development	Medical	Female	Research Scientist	Single	No
275	No	Research & Development	Medical	Female	Research Director	Divorced	Yes

Figure 5: Random Sample of dataset with only Categorical Feature

Specifically, the results table includes the following columns:

- **Attrition:** Status of leaving work (Yes/No).
- **Department:** Working department (Research & Development, Sales, ...).
- **EducationField:** Education field (Medical, Life Sciences, ...).
- **Gender:** Gender (Male/Female).
- **JobRole:** Job role (Research Director, Sales Executive, ...).

- **MaritalStatus:** Marital status (Married, Single, Divorced).
- **OverTime:** Working overtime (Yes/No).

Some of the numerical features are storing discrete numbers (categories labelled) in numbers. So for better analysis we will replace those labelled numerical values with appropriate categorical values.

```
mappings = {
    "EnvironmentSatisfaction": {"Low": 1, "Medium": 2, "High": 3, "Very High": 4},
    "JobInvolvement": {"Low": 1, "Medium": 2, "High": 3, "Very High": 4},
    "JobSatisfaction": {"Low": 1, "Medium": 2, "High": 3, "Very High": 4},
    "RelationshipSatisfaction": {"Low": 1, "Medium": 2, "High": 3, "Very High": 4},
    "WorkLifeBalance": {"Bad": 1, "Good": 2, "Better": 3, "Best": 4},
    "Education": {"Below College": 1, "College": 2, "Bachelor": 3, "Master": 4, "Doctor": 5}
}

for col, mapping in mappings.items():
    data[col] = data[col].replace(mapping)
```

Figure 6: Mapping

Converts categorical values in some columns of DataFrame **'data'** to numeric values based on **'mappings'** defined in the mappings dictionary. This makes it easier to analyze data or implement machine learning models because the models often require numerical input.

- The **EnvironmentSatisfaction** column with the values "Low", "Medium", "High", "Very High" will be replaced with the corresponding values 1, 2, 3, 4.
- The **Education** column with the values "Below College", "College", "Bachelor", "Master", "Doctor" will be replaced with the corresponding values 1, 2, 3, 4, 5.

```
[ ] #Random Sample of dataset with only categorical Feature
    data.select_dtypes(include='O').sample(5)
```

	Attrition	Department	EducationField	Gender	JobRole	MaritalStatus	OverTime
638	No	Sales	Marketing	Male	Sales Executive	Married	No
640	No	Research & Development	Life Sciences	Male	Laboratory Technician	Married	No
958	No	Research & Development	Life Sciences	Male	Healthcare Representative	Divorced	No
163	No	Research & Development	Life Sciences	Male	Healthcare Representative	Divorced	Yes
907	No	Sales	Marketing	Male	Manager	Married	No

Figure 7: Random Sample of dataset with only Numerical Feature

2.2. Data Cleaning

- Check the missing value: There is no missing data

	Total No. of Missing Values	% of Missing Values
Age	0	0.0
Attrition	0	0.0
BusinessTravel	0	0.0
DailyRate	0	0.0
Department	0	0.0
DistanceFromHome	0	0.0
Education	0	0.0
EducationField	0	0.0
EmployeeCount	0	0.0
EmployeeNumber	0	0.0
EnvironmentSatisfaction	0	0.0
Gender	0	0.0
HourlyRate	0	0.0
JobInvolvement	0	0.0
JobLevel	0	0.0
JobRole	0	0.0
JobSatisfaction	0	0.0
MaritalStatus	0	0.0
MonthlyIncome	0	0.0
MonthlyRate	0	0.0
NumCompaniesWorked	0	0.0
Over18	0	0.0
OverTime	0	0.0
PercentSalaryHike	0	0.0
PerformanceRating	0	0.0
RelationshipSatisfaction	0	0.0
StandardHours	0	0.0
StockOptionLevel	0	0.0
TotalWorkingYears	0	0.0

Figure 8: Missing value

- Check for duplicates

```
#Checking if There's Any Duplicate Records.
print("Duplicates in Dataset: ",data.duplicated().sum())
```

```
Duplicates in Dataset:  0
```

Figure 9: Duplicate records

- Check the unique value

```
#Checking Unique Values of Categorical Attributes.
cat_cols = data.select_dtypes(include="object").columns

for column in cat_cols:
    print('Unique values of ', column, set(data[column]))
    print("-"*121)
```

```
Unique values of  Attrition {'No', 'Yes'}
-------------------------------------------------------------------------------------------------------------------------
Unique values of  BusinessTravel {'Travel_Rarely', 'Travel_Frequently', 'Non-Travel'}
-------------------------------------------------------------------------------------------------------------------------
Unique values of  Department {'Sales', 'Human Resources', 'Research & Development'}
-------------------------------------------------------------------------------------------------------------------------
Unique values of  EducationField {'Marketing', 'Technical Degree', 'Other', 'Life Sciences', 'Medical', 'Human Resources'}
-------------------------------------------------------------------------------------------------------------------------
Unique values of  Gender {'Female', 'Male'}
-------------------------------------------------------------------------------------------------------------------------
Unique values of  JobRole {'Research Director', 'Laboratory Technician', 'Sales Executive', 'Healthcare Representative', 'Research Scientist', 'Manager', 'Sales Representative',
-------------------------------------------------------------------------------------------------------------------------
Unique values of  MaritalStatus {'Married', 'Divorced', 'Single'}
-------------------------------------------------------------------------------------------------------------------------
Unique values of  Over18 {'Y'}
-------------------------------------------------------------------------------------------------------------------------
Unique values of  OverTime {'No', 'Yes'}
-------------------------------------------------------------------------------------------------------------------------
```

Figure 10: The unique value

- Scaling the data

```
[ ] scaler = StandardScaler()
```

```
[ ] x_scaled = scaler.fit_transform(x_smote)
    data_scaled = scaler.fit_transform(data)
```

Figure 11: Features scaling

2.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an essential step in the data analysis process, where we use various statistical and graphical techniques to understand the structure, relationships, and patterns within a dataset. EDA helps to summarize the main characteristics of the data, identify trends, detect anomalies, test hypotheses, and check assumptions.

The dataset consists of 1,470 observations and 21 variables, providing detailed information about employees such as age, gender, department, distance from home, job satisfaction level, involvement level, and more. This is the initial step in the Exploratory Data Analysis (EDA) process, aiming to examine the data structure, identify variable types, and preliminarily detect any anomalies or missing values.

```
[ ] data = pd.read_csv('/content/datafullreal.csv')
    data.head()
```

	Age	Attrition	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfaction	Gender	HourlyRate	JobInvolvement	...	JobSatisfaction
0	41	Yes	Sales	1	2	Life Sciences	2	Female	94	3	...	4
1	49	No	Research & Development	8	1	Life Sciences	3	Male	61	2	...	2
2	37	Yes	Research & Development	2	2	Other	4	Male	92	2	...	3
3	33	No	Research & Development	3	4	Life Sciences	4	Female	56	3	...	3
4	27	No	Research & Development	2	1	Medical	1	Male	40	3	...	2

5 rows × 21 columns

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
[ ] data.shape
```

```
(1470, 21)
```

Figure 12: Dataset Preview and Dimension Summary

The dataset consists of 1,470 entries and 21 features, with no missing values across any columns. There are 14 numerical features with int64 data type, including variables such as Age, MonthlyIncome, and YearsAtCompany, which are mostly continuous or ordinal in nature. The remaining 7 features are categorical (object type), such as Department, Gender, JobRole, and the target variable Attrition, which indicates whether an employee has left the company. The clean structure and balanced mix of numerical and categorical features make this dataset well-suited for exploratory data analysis (EDA) and predictive modeling tasks, particularly classification problems.

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 21 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   Department                1470 non-null   object
 3   DistanceFromHome          1470 non-null   int64
 4   Education                 1470 non-null   int64
 5   EducationField            1470 non-null   object
 6   EnvironmentSatisfaction   1470 non-null   int64
 7   Gender                    1470 non-null   object
 8   HourlyRate                1470 non-null   int64
 9   JobInvolvement            1470 non-null   int64
 10  JobRole                   1470 non-null   object
 11  JobSatisfaction           1470 non-null   int64
 12  MaritalStatus             1470 non-null   object
 13  MonthlyIncome             1470 non-null   int64
 14  OverTime                  1470 non-null   object
 15  PercentSalaryHike         1470 non-null   int64
 16  RelationshipSatisfaction  1470 non-null   int64
 17  TotalWorkingYears         1470 non-null   int64
 18  WorkLifeBalance           1470 non-null   int64
 19  YearsAtCompany            1470 non-null   int64
 20  YearsInCurrentRole        1470 non-null   int64
dtypes: int64(14), object(7)
memory usage: 241.3+ KB
```

Figure 13: Data information

The "Distribution of Age & YearsAtCompany by Attrition" chart reveals key insights. Employees who attrit (red curve for Age, orange for YearsAtCompany) tend to be younger and, more significantly, have shorter tenures at the company. Their age distribution peaks around 25-35, while their years at company distribution peaks sharply at very low values (under 5 years). Conversely, employees who do not attrit (blue curve for Age, green for YearsAtCompany) are generally older (peak around 35-45) and, more notably, have much longer tenures. This suggests that while age plays a role, the number of years an employee has been with the company is a much stronger predictor of attrition.

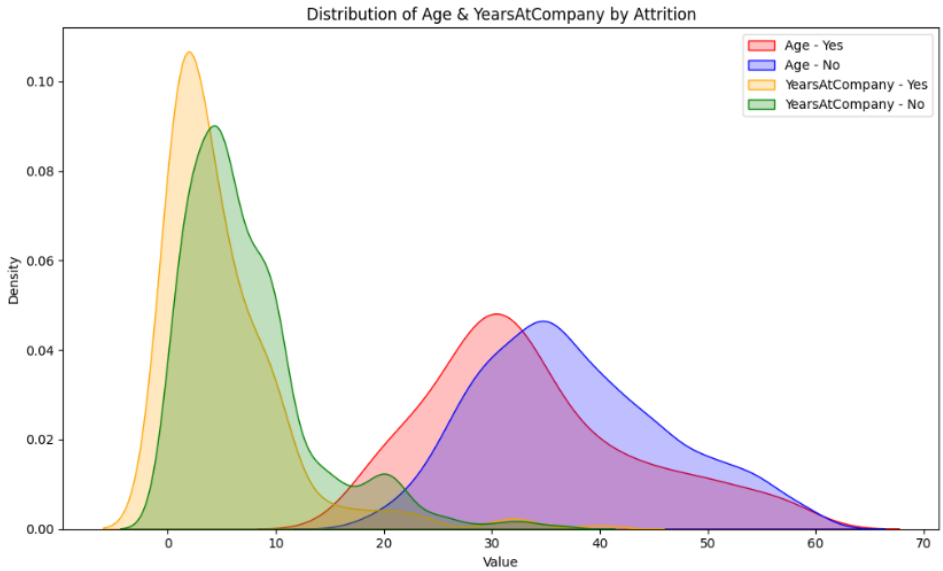


Figure 14: Density Distribution of Age and Years at Company by Attrition Status

Between the predictors, 'MonthlyIncome' is highly correlated with 'JobLevel' (0.95), suggesting that these variables might be conveying overlapping information. 'TotalWorkingYears' has strong positive correlations with 'JobLevel' (0.78) and 'MonthlyIncome' (0.77), which is expected as more experienced employees are likely at a higher job level and earn more.

For this analysis, I will remove MonthlyIncome, TotalWorkingYears and YearsInCurrentRole using a cutoff of 0.7 correlation coefficient. This will retain JobLevel and YearsAtCompany and remove the possibility of multicollinearity from the features.

For MonthlyIncome, the high correlation with job level and sensitivity around salary makes it both redundant and hard to work within policy change management. For TotalWorkingYears, the data will be encapsulated in other features like YearsAtCompany. For YearsInCurrentRole the data will not pose much additional insights beyond YearsAtCompany, considering organizational structures/functional specializations (e.g. Accounting) are what keep individuals in similar Department and role.

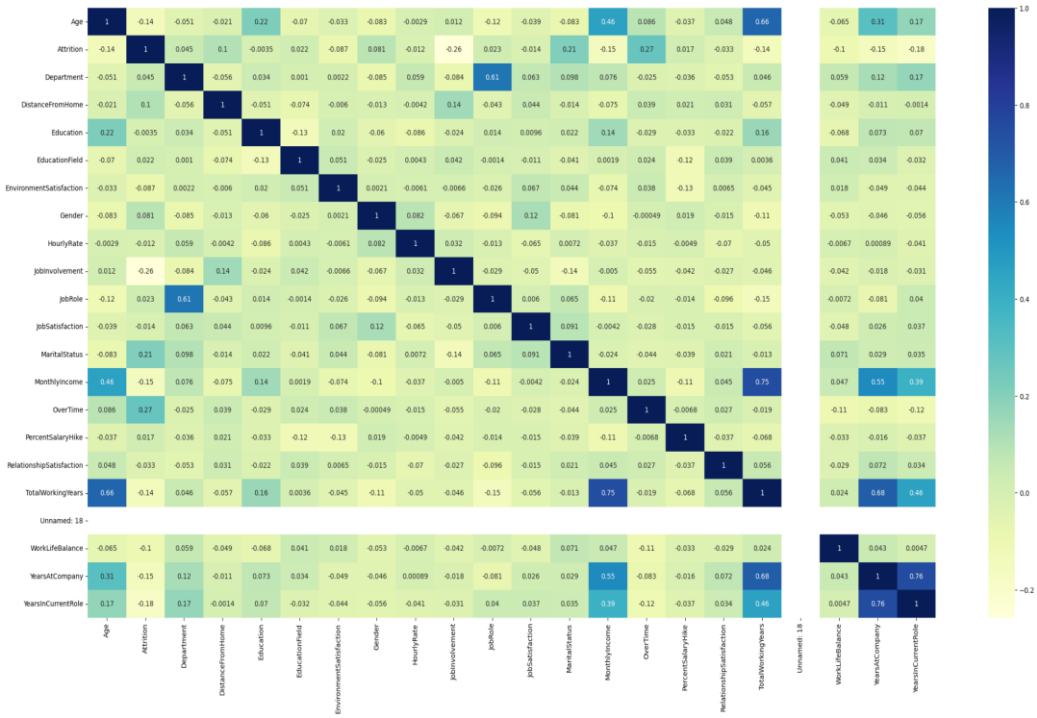


Figure 15: Heatmap

A summary of descriptive statistics for a dataset, obtained using the df.describe() function. It provides key statistical measures for several numerical columns. For all listed features, the count is consistently 1470, indicating that there are no missing values across these attributes. The table includes common metrics such as the mean, standard deviation (std), minimum (min), 25th percentile, 50th percentile (median), 75th percentile, and maximum (max) values. Looking at specific columns, we can observe that EmployeeCount has a constant value of 1.0 for all entries, as shown by its identical min, max, and mean values and a standard deviation of 0.0. EmployeeNumber spans a wide range from 1 to 2068, with a mean of approximately 1024.8. Other columns like Age, DailyRate, DistanceFromHome, Education, HourlyRate, and JobInvolvement also have their respective statistics presented, offering a quick overview of their central tendency, spread, and range within the dataset.

```
df.describe()
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	HourlyRate	JobInvolvement
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	1470.000000	1470.000000	1470.000000	1470.000000
mean	36.923810	802.485714	9.192517	2.912925	1.0	1024.865306	2.721769	65.891156	2.729932
std	9.135373	403.509100	8.106864	1.024165	0.0	602.024335	1.093082	20.329428	0.711561
min	18.000000	102.000000	1.000000	1.000000	1.0	1.000000	1.000000	30.000000	1.000000
25%	30.000000	465.000000	2.000000	2.000000	1.0	491.250000	2.000000	48.000000	2.000000
50%	36.000000	802.000000	7.000000	3.000000	1.0	1020.500000	3.000000	66.000000	3.000000
75%	43.000000	1157.000000	14.000000	4.000000	1.0	1555.750000	4.000000	83.750000	3.000000
max	60.000000	1499.000000	29.000000	5.000000	1.0	2068.000000	4.000000	100.000000	4.000000

Figure 16: Data describe

2.4. Data preprocessing

a. *Check for imbalance in dataset*

The bar chart "Employee Attrition Counts," indicates the absolute numbers of employees

who did not attrit (represented by '0') and those who did (represented by '1'). It clearly shows that a vast majority of employees, 1233 individuals, remained with the company, while 237 employees attrited. Complementing this, the donut chart "Employee Attrition Rate," visualizes these numbers as percentages. It highlights that the overwhelming majority, 83.88%, of employees chose to stay, while a smaller but significant portion, 16.12%, represents the attrition rate. These two charts collectively demonstrate that while attrition does occur, the retention rate within the dataset is notably high.

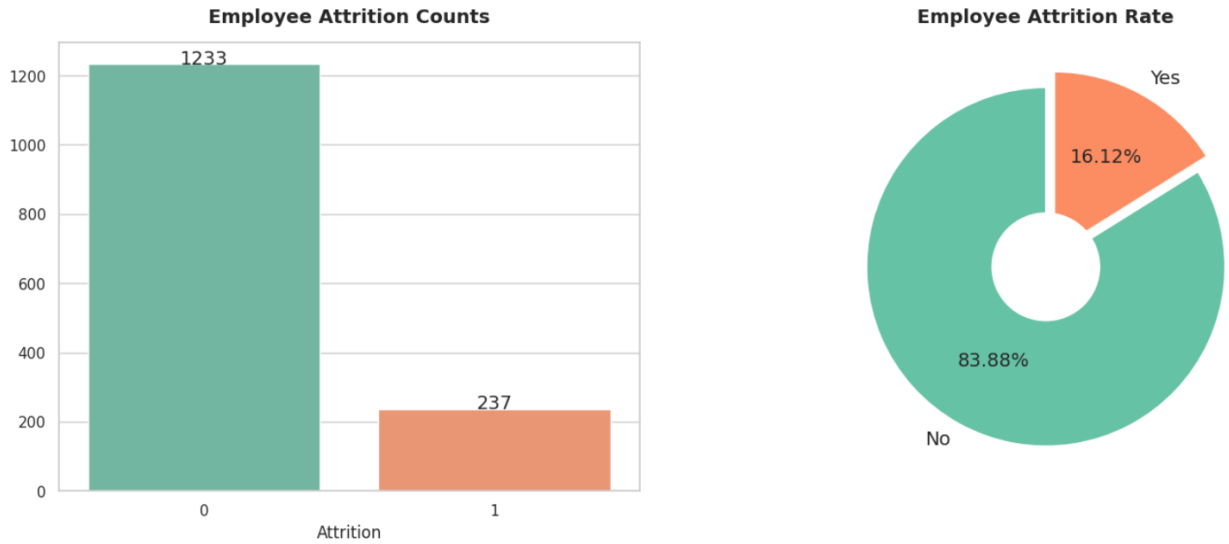


Figure 17: Visualization 'Attrition'

b. *Split the data*

```
[ ] x = data.drop(['Attrition'], axis=1)
    y = data[['Attrition']]
```

```
[ ] x.head()
```

	Age	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfaction	Gender	HourlyRate	JobInvolvement	JobRole	JobSatisfaction
0	41	2	1	1	1	1	0	94	2	7	3
1	49	1	8	0	1	2	1	61	1	6	1
2	37	1	2	1	4	3	1	92	1	2	2
3	33	1	3	3	1	3	0	56	2	6	2
4	27	1	2	0	3	0	1	40	2	2	1

```
y.head()
```

	Attrition
0	1
1	0
2	1
3	0
4	0

Figure 18: Separate dependent and independent variables

c. *Handle data imbalance use SMOTE*

Prior to model building, we will split the data into training and test dataset and balance the data using SMOTE (Synthetic Minority Over-sampling Technique) due to the highly imbalanced

class distribution in Attrition

```
#Unscaled
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_smote, y_smote, test_size=0.2, random_state=42)
print(x_train.shape, y_train.shape, x_test.shape, y_test.shape)
```

```
(1972, 15) (1972,) (494, 15) (494,)
```

```
#Scaled
x_train1, x_test1, y_train1, y_test1 = train_test_split(x_scaled, y_smote, test_size=0.2, random_state=42)
print(x_train1.shape, y_train1.shape, x_test1.shape, y_test1.shape)
```

```
(1972, 15) (1972,) (494, 15) (494,)
```

Figure 19: Training and test data

3. Methodology

3.1. Workflow

This flowchart outlines a standard machine learning workflow. It starts with the **Dataset**, which is then subjected to **Data Processing** and **EDA (Exploratory Data Analysis)**. Following these preparatory steps, a **Built Model** phase involves training various algorithms, specifically **KNeighborsClassifier**, **Random Forest**, **Gradient Boosting**, and a **Stacking Model**. The outputs from these models are then used to generate the final **Result**.

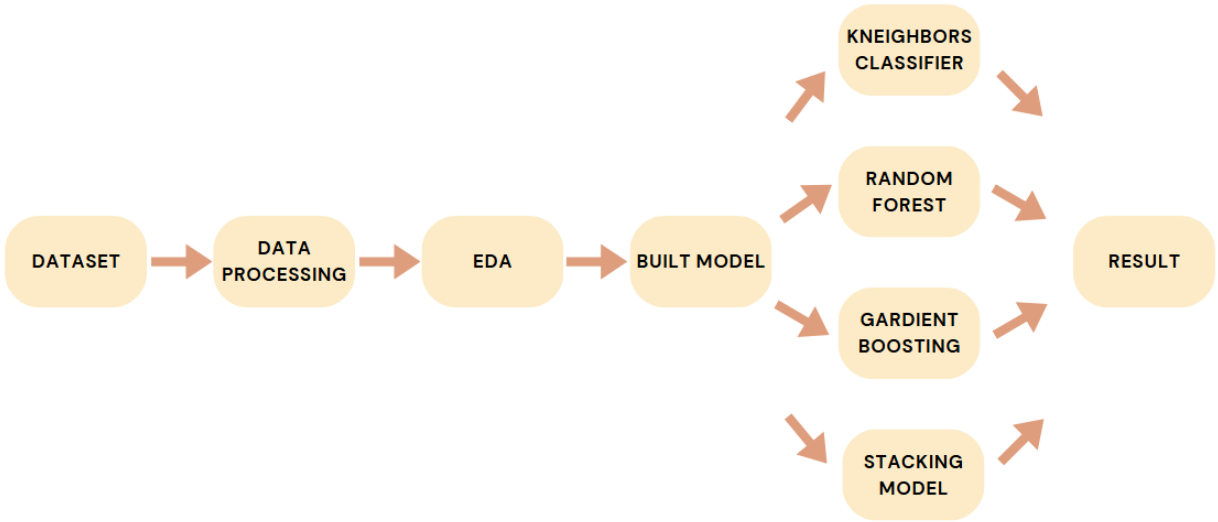


Figure 20: Flow chart

3.2. Techniques Applied

In the process of building the employee attrition prediction model, several data preprocessing techniques and machine learning algorithms were applied to improve accuracy and model generalization. Initially, the SMOTE (Synthetic Minority Oversampling Technique) method was used to balance the training dataset. Since the original data showed class imbalance (employees who left vs. those who stayed), SMOTE helped augment the minority class by generating synthetic samples based on existing data points. This approach reduced model bias during

training.

Subsequently, StandardScaler was employed to normalize continuous numerical features. Standardization ensures that all features are on the same scale, which is particularly important for algorithms that are sensitive to feature magnitude, such as K-Nearest Neighbors (KNN) and Logistic Regression.

The core prediction method implemented was a Stacking Ensemble model. This technique combines predictions from multiple base classifiers (Random Forest, KNN, and Gradient Boosting) using a meta-classifier (Logistic Regression) to produce the final output. Stacking leverages the strengths of individual models and often yields better overall performance due to its ability to learn from diverse perspectives within the data.

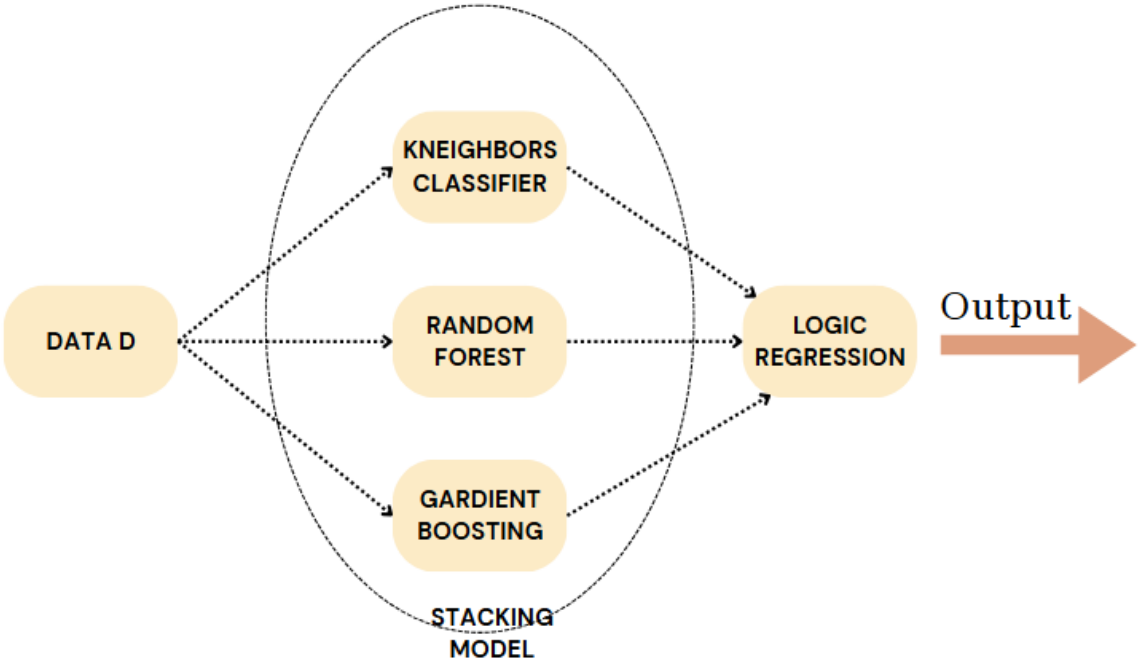


Figure 21: Stacking model

To evaluate the model, several performance metrics were used. The ROC AUC Score was calculated to assess the model's ability to distinguish between the two classes. The F1-Score, which balances precision and recall, was particularly valuable for this imbalanced classification task. Additionally, a Confusion Matrix was generated to clearly observe the distribution of true positives, true negatives, false positives, and false negatives.

```python
from sklearn.ensemble import RandomForestClassifier


param_grid = {
    'max_depth': [20],
    'n_estimators': [220],
    'min_samples_split': [30],
    'min_samples_leaf': [10],
    'max_features': ["sqrt"]
}

model_random_forest = RandomForestClassifier(random_state=42)

model_random_forest_grid_search = GridSearchCV(model_random_forest, param_grid=param_grid, cv=5, scoring='recall', n_jobs=-1)

model_random_forest_grid_search.fit(X_train_scaled, y_over)

y_pred = model_random_forest_grid_search.predict(X_test_scaled)


print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot = True, fmt = "g")
```

Figure 22: Random forest on dataset

```python
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, roc_auc_score # Import roc_curve
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming X_train_scaled, y_over, X_test_scaled, y_test are already defined

# Initialize the K-Nearest Neighbors model
model_knn_tuning = KNeighborsClassifier()

# Xây dựng Grid Search với param_grid cho KNN
param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 13],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

# NOTE: You were using model_xgboost_tuning as the estimator for GridSearchCV for KNN.
# This should be model_knn_tuning. Corrected below.
grid_search = GridSearchCV(estimator=model_knn_tuning, param_grid=param_grid, cv=5, scoring='recall', n_jobs=-1)
grid_search.fit(X_train_scaled, y_over)

model_knn_tuning = grid_search.best_estimator_
y_pred = model_knn_tuning.predict(X_test_scaled)

print("--- Classification Report for Tuned KNN Model ---")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
# Changed 'cmap' to 'rocket' for a darker pink/purple gradient
sns.heatmap(cm, annot=True, fmt="g", cmap="rocket",
            xticklabels=['Predicted Negative', 'Predicted Positive'],
            yticklabels=['Actual Negative', 'Actual Positive'])
plt.title('Confusion Matrix for K-Nearest Neighbors')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Figure 23: KNeighborsClassifier on dataset

3.3. Tools and Libraries Used

The entire project was implemented using the Python programming language, which offers a rich ecosystem of data science libraries. The pandas library was utilized for data manipulation and preprocessing, while numpy supported numerical computations and array handling. For data visualization and interpretation of results, matplotlib and seaborn were employed to create

informative plots and charts. In terms of machine learning modeling and evaluation, the scikit-learn (sklearn) library served as the backbone, providing tools for preprocessing, model training, and performance measurement. The imblearn library was integrated to apply SMOTE and address class imbalance effectively. Finally, openpyxl was used to export prediction results to Excel format, facilitating reporting and post-analysis.

To deploy the model on the web, we use Streamlit. Streamlit applications have gained popularity in both the data science and web development communities because they allow for the creation of interactive user interfaces quickly and easily. In this report, we will outline the steps involved in setting up the Streamlit application using the function provided below. First, we import the pickle library, which helps in saving and loading complex data such as Python objects from files. We also import the Streamlit library and assign it a shorthand name for convenience in later steps. Streamlit is a powerful tool for rapidly building web applications that are focused on data science tasks.

4. Results

4.1. Random Forest model

Table 2: Result of random forest model

	precision	recall	f1-score	support
0	0.86	0.85	0.85	203
1	0.49	0.52	0.50	58
accuracy			0.77	261
macro avg	0.68	0.68	0.68	261
weighted avg	0.78	0.77	0.78	261

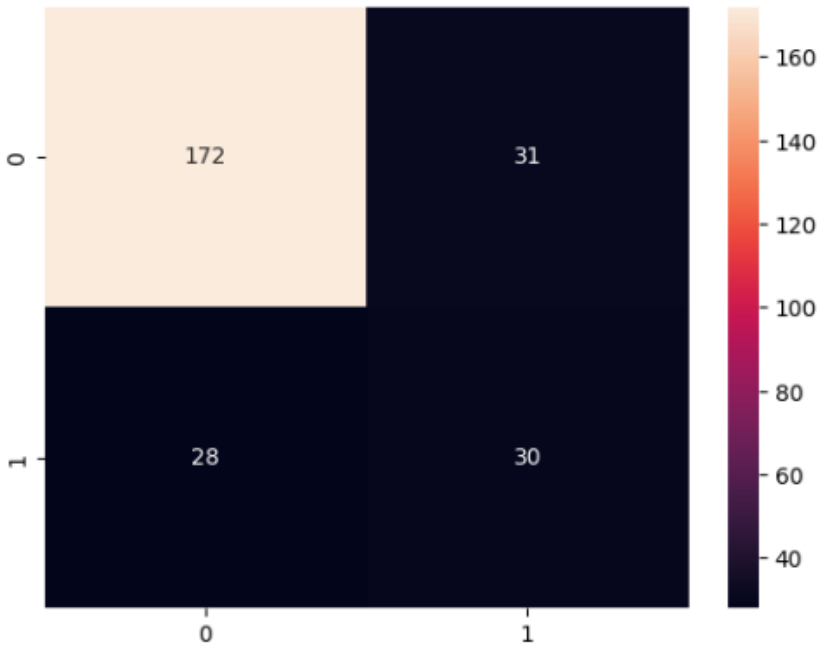


Figure 24: Random Forest

4.2. XGBoost model

Table 3: Results of XGBoost model

	precision	recall	f1-score	support
0	0.85	0.95	0.90	203
1	0.69	0.41	0.52	58
accuracy			0.83	261
macro avg	0.77	0.68	0.71	261
weighted avg	0.81	0.83	0.81	261

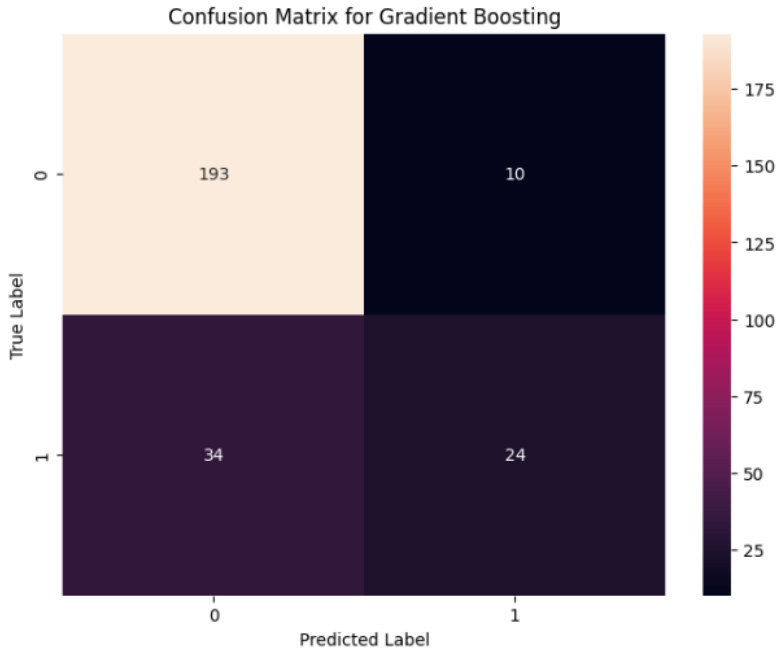


Figure 25: XGBoost

4.3. KNeighbors Classifier model

Table 4: Results of KNeighbors Classifier model

	precision	recall	f1-score	support
0	0.86	0.75	0.80	203
1	0.40	0.57	0.47	58
accuracy			0.71	261
macro avg	0.63	0.66	0.64	261
weighted avg	0.76	0.71	0.73	261

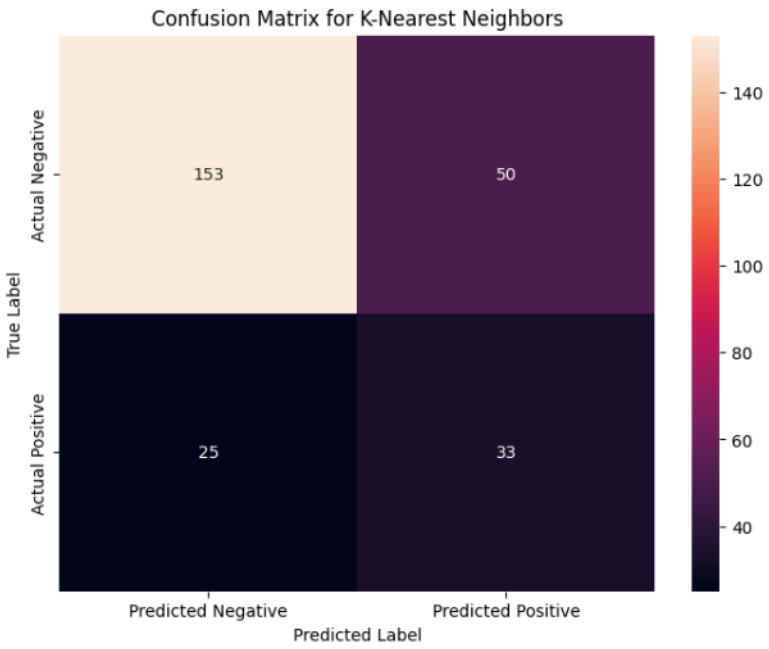


Figure 26: KNeighbors Classifier

4.4. Stacking model

Table 5: Results of Stacking model

	precision	recall	f1-score	support
0	0.82	0.99	0.90	203
1	0.87	0.22	0.36	58
accuracy			0.82	261
macro avg	0.84	0.61	0.63	261
weighted avg	0.83	0.82	0.78	261

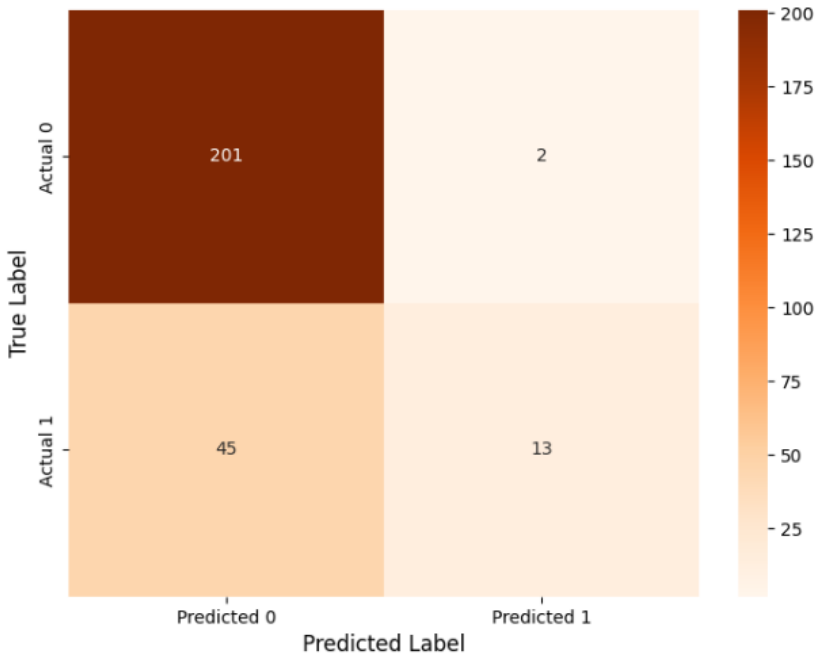


Figure 27: Stacking model

4.5. Result summary

Table 6: Result summary

Model	Precision	Recall	F1-Score	Accuracy
Random Forest	0.86	0.85	0.85	0.77
XGBoost	0.85	0.95	0.90	0.83
KNeighbors Classifier	0.86	0.75	0.80	0.71
Stacking model	0.82	0.99	0.90	0.82

Stacking model demonstrably exhibited the most superior performance across key metrics, achieving the highest Precision and a highly competitive Accuracy. In contrast, the individual Random Forest, XGBoost, and KNeighbors Classifier models did not achieve comparable levels of superiority, with KNeighbors Classifier showing the weakest overall performance. This indicates that the ensemble approach of stacking effectively leveraged the strengths of its base models to yield the most robust and accurate predictions for this specific attrition problem.

4.6. Application

Actual Employee Attrition Counts illustrates the distribution of employee attrition within a dataset. It clearly shows that 254 employees did not attrite, while 98 employees did attrite. The significant disparity between these two numbers indicates that the majority of employees have been retained, suggesting a relatively low attrition rate in this specific context.

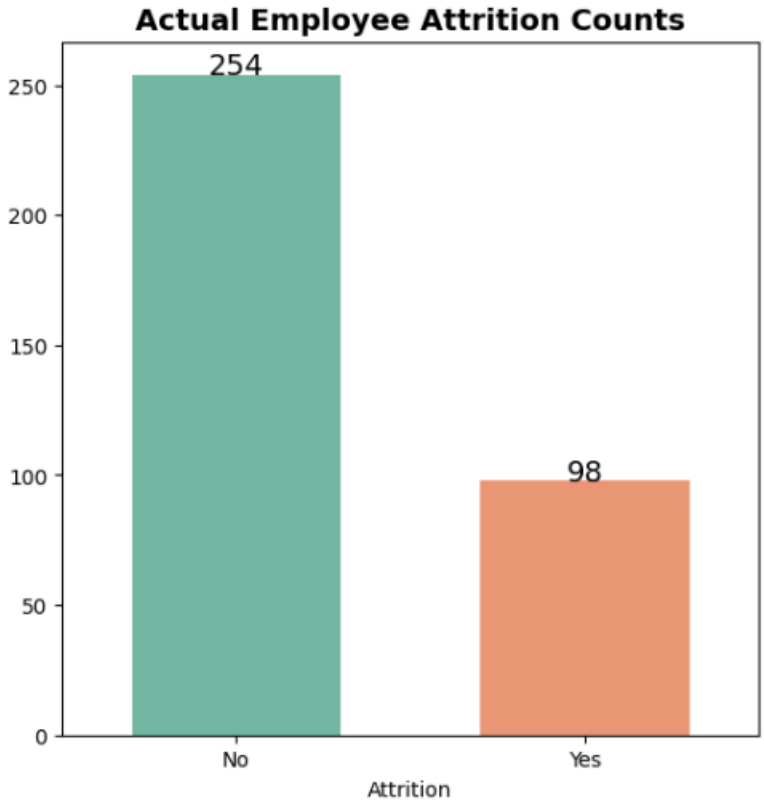
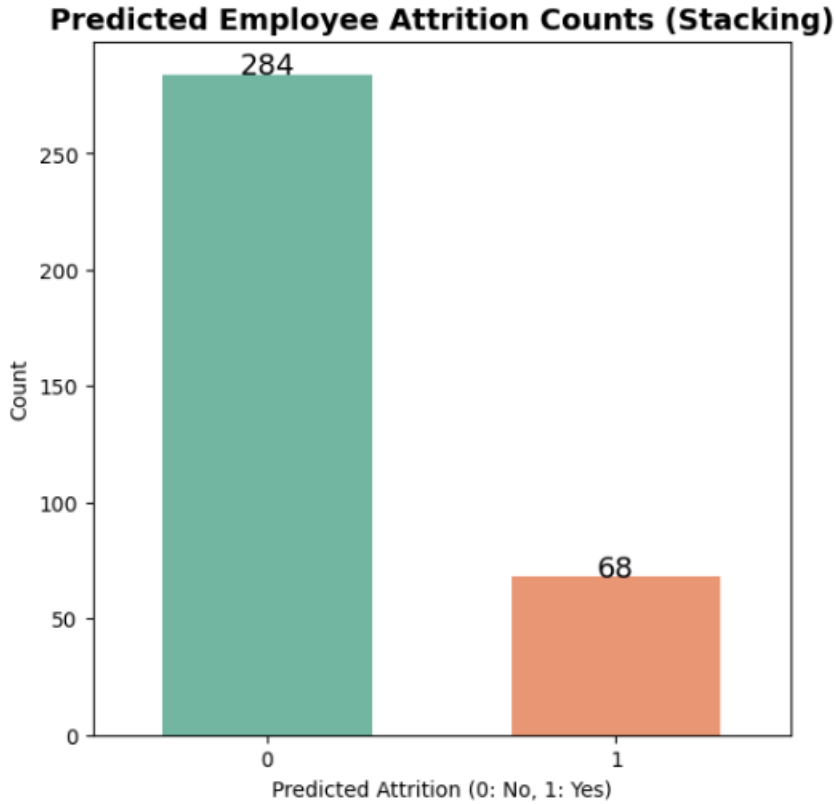


Figure 28: Actual employee attrition counts

Predicted Employee Attrition Counts (Stacking) displays the output of a predictive model (likely a "Stacking" ensemble model) for employee attrition. The '0' category represents 284 employees predicted to not attrite, while the '1' category represents 68 employees predicted to attrite. Similar to the actual counts, there's a notable imbalance in the predictions, with a much larger number of employees predicted to stay. Crucially, the chart also provides an ROC AUC Score of 0.9660. This is a very high score (close to 1.0), indicating that the stacking model demonstrates excellent discriminatory power in distinguishing between employees who will attrite and those who will not. In simpler terms, the model is highly effective at making accurate predictions regarding employee attrition.



ROC AUC Score: 0.9660

Figure 29: Predicted employee atrition counts (Stracking)

5. Conclusion

Given that this dataset was collected from IBM, IBM Company and IBM Human Resources are the primary stakeholders who need to comprehend the insights provided by our report to enhance recruitment strategies and human resource management.

Based on the model outcomes, the following observations are drawn:

- Employees often undergo career changes at the beginning or early stages of their professional journey, seeking roles that align better with their evolving aspirations and skill sets.
- Long-term organizational loyalty is prevalent among employees who have achieved stability in their personal lives or have secured stable employment within the company. These individuals typically progress vertically through the organization, driven by internal opportunities for growth and development.
- Financial incentives, including competitive salaries and stock options, significantly influence employee loyalty. Enhanced compensation packages are associated with higher retention rates, underscoring the importance of aligning financial rewards with employee satisfaction and performance.
- Work-life balance emerges as a critical motivator for employees. Those able to maintain a healthy balance between work and personal life are more likely to remain content

within their current roles. Conversely, individuals perceiving an imbalance are prone to exploring external opportunities that promise better prospects and quality of life.

- Departments focused on achieving specific performance metrics, such as sales, often experience higher turnover rates compared to administrative functions like Human Resources. This discrepancy highlights the varying motivational factors and career trajectories across different organizational units.
- Job satisfaction and workplace environment significantly impact employee retention rates. Satisfied employees are more inclined to exhibit loyalty towards their organization, whereas dissatisfied individuals are more likely to seek alternative employment opportunities that better align with their professional and personal aspirations.

The stacking ensemble model, integrating Random Forest, K-Nearest Neighbors, and XG Boosting with Logistic Regression as the meta-classifier, demonstrated reasonable performance in predicting employee attrition. On the actual test set, the model achieved high precision for the majority class (employees who stayed), correctly predicting 201 out of 203 cases. However, it correctly identified only 13 of the 58 attrition cases, resulting in 45 false negatives. This indicates a strong predictive power for employee retention but limited sensitivity in detecting actual leavers a common challenge in imbalanced classification problems. While overall accuracy remains acceptable, the misclassification of employees who actually left is critical from a business perspective. These false negatives represent missed opportunities for early intervention by HR departments. The model's predicted probabilities show low confidence for many of these leavers, suggesting the need for deeper feature engineering or alternative modeling strategies. Nonetheless, the model provides a useful foundation for initial attrition screening and opens the door for proactive employee engagement strategies.

- Limitations and Challenges: Key challenges encountered include the persistent class imbalance in the test set, even after applying SMOTE to the training data. Although SMOTE helps balance training, it does not influence the real-world distribution of attrition, which is naturally skewed. This leads to biased learning that favors the majority class. Moreover, the current feature set may not fully capture behavioral or psychological aspects influencing employee resignation, limiting the model's discriminative ability.
- Recommendations for Further Research or Application: Future enhancements should focus on incorporating additional data sources such as employee surveys, sentiment analysis from feedback platforms, performance trends, and exit interviews. Advanced modeling techniques like XGBoost with cost-sensitive loss functions, anomaly detection for rare-event modeling, or semi-supervised learning can be explored to improve minority class

detection. Additionally, threshold tuning based on business risk tolerance and model interpretability techniques (e.g., SHAP values) can help translate predictions into actionable insights for HR professionals. Ultimately, while this predictive framework is not yet a definitive attrition predictor, it forms a robust analytical baseline to guide further development and HR decision-making.

6. References

[1]. Viblo, 2024. *Logistic Regression - Bài toán cơ bản trong Machine Learning*. [online] Available at: https://viblo.asia/p/logistic-regression-bai-toan-co-ban-trong-machine-learning-924lJ4rzKPM [Accessed 4 December 2024].

[2]. Machine Learning Cơ Bản, 2017. *Logistic Regression*. [online] Available at: https://machinelearningcoban.com/2017/01/27/logisticregression/ [Accessed 4 December 2024].

[3]. Machine Learning Cơ Bản, 2017. *Naive Bayes Classifier (NBC)*. [online] Available at: https://machinelearningcoban.com/2017/08/08/nbc/ [Accessed 4 December 2024].

[4]. Viblo, 2024. *TF-IDF - Term Frequency Inverse Document Frequency*. [online] Available at: https://viblo.asia/p/tf-idf-term-frequency-inverse-document-frequency-JQVkVZgKkyd [Accessed 4 December 2024].

[5]. Pham Dinh Khanh, 2020. *Confusion Matrix: Đánh giá mô hình học máy*. [online] Available at: https://phamdinhkhanh.github.io/2020/08/13/ModelMetric.html [Accessed 4 December 2024].

[6]. Huang, Y., Huang, B., & Kechadi, M. T. (2011). A rule-based method for customer churn prediction in telecommunication services. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, *6634 LNAI*(PART 1), 411–422.

[7]. Jain, H., Khunteta, A., & Srivastava, S. (2020). Churn Prediction in Telecommunication using Logistic Regression and Logit Boost. *Procedia Computer Science*, *167*, 101–112.

[8]. Kim, H. S., & Yoon, C. H. (2004). Determinants of subscriber churn and customer loyalty in the Korean mobile telephony market. *Telecommunications Policy*, *28*(9–10), 751–765.

[9]. Đỗ Trung Tuấn, 1997. *Trí Tuệ Nhân Tạo (NXB Giáo Dục 1997)* [online]. Available at: https://vietbooks.info/threads/tri-tue-nhan-tao-nxb-giao-duc-1997-do-trung-tuan-487-trang.105041/